Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: Aguilar Ardila Gabriel Esteban **Fecha:** 22/02/2026 **Asignatura:** Estructura de Computadores

Enlace del repositorio en GitHub: [agregar enlace Aquí]

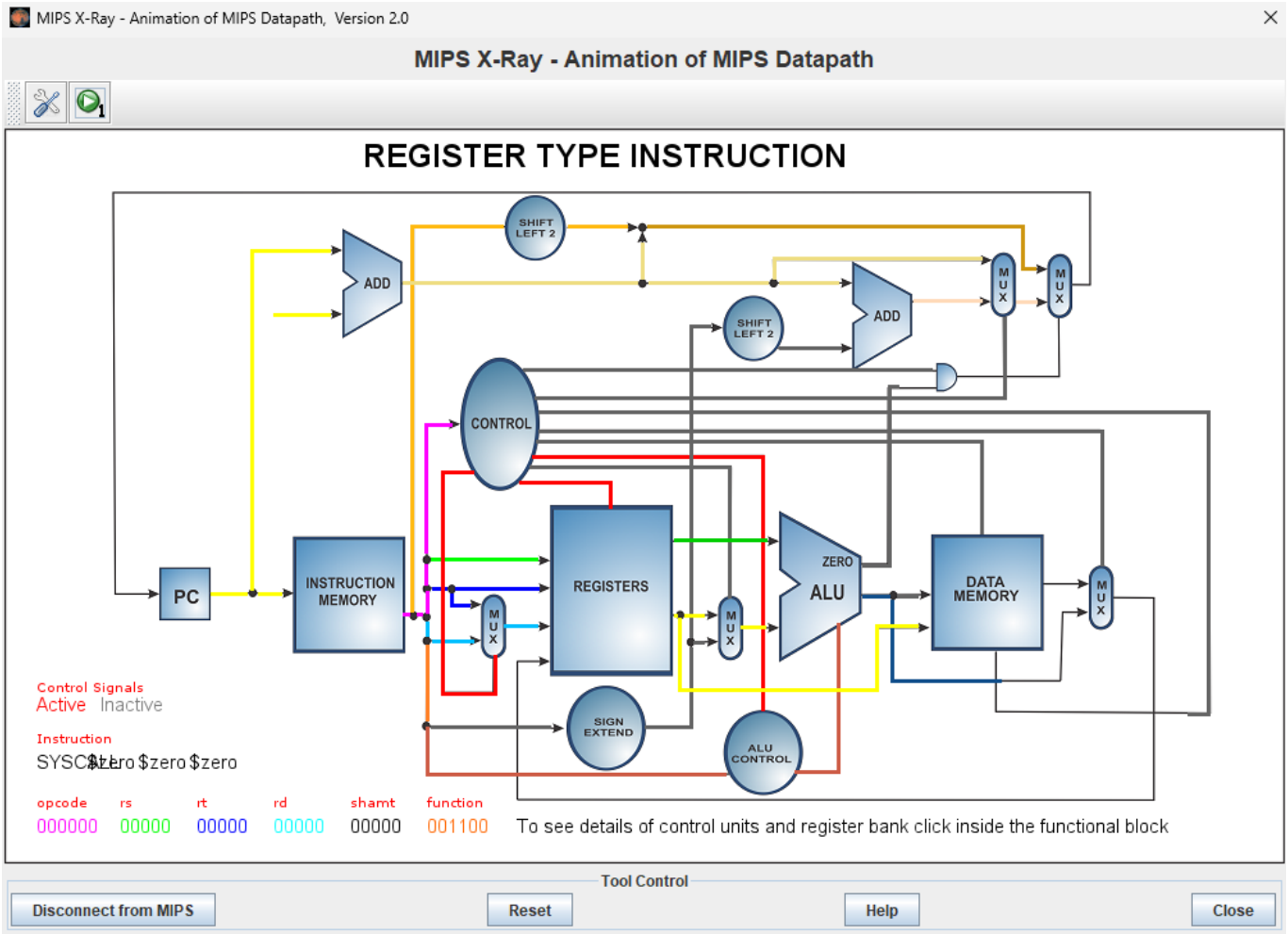
---

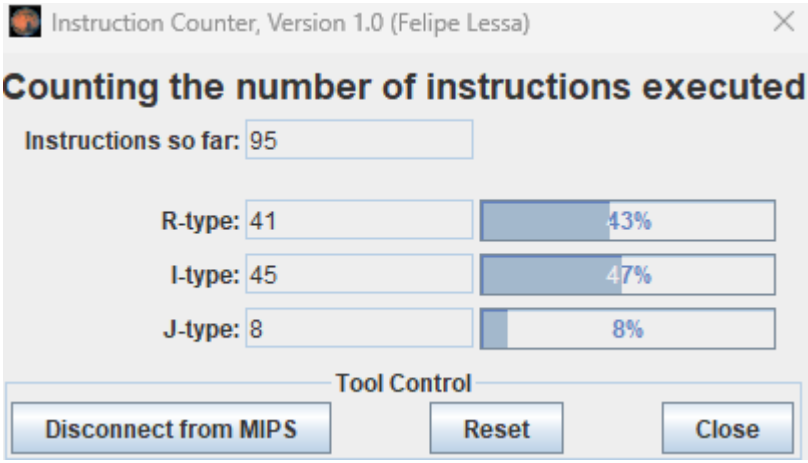
1. Análisis del Código Base

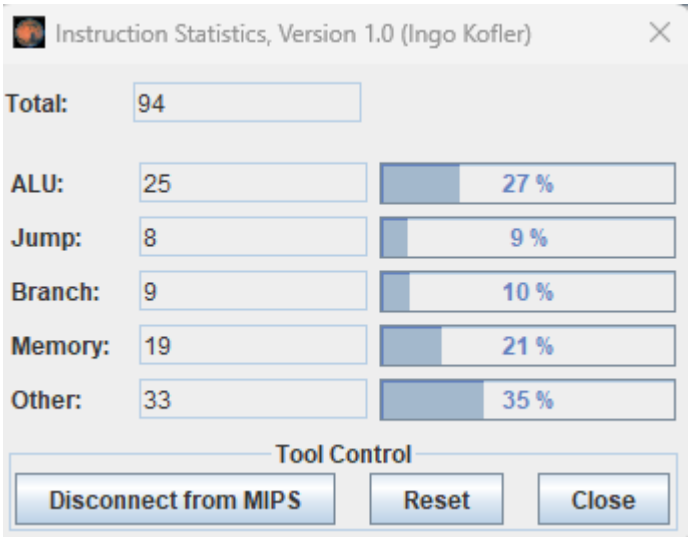
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

- **MIPS X-Ray** (Ventana con el Datapath animado).
- **Instruction Counter** (Contador de instrucciones totales).
- **Instruction Statistics** (Desglose por tipo de instrucción).


MIPS X-Ray - Animation of MIPS Datapath, Version 2.0
MIPS X-Ray - Animation of MIPS Datapath
REGISTER TYPE INSTRUCTION
SHIFT LEFT 2
ADD
ADD
SHIFT LEFT 2
MUX
MUX
CONTROL
PC
INSTRUCTION MEMORY
REGISTERS
ZERO
ALU
DATA MEMORY
MUX
MUX
MUX
SIGN EXTEND
ALU CONTROL
Control Signals
Active Inactive
Instruction
SYSCALL $zero $zero $zero
opcode rs rt rd shamt function
000000 00000 00000 00000 00000 001100
To see details of control units and register bank click inside the functional block
Tool Control
Disconnect from MIPS
Reset
Help
Close


Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 95
R-type: 41
43%
I-type: 45
47%
J-type: 8
8%
Tool Control
Disconnect from MIPS
Reset
Close


Instruction Statistics, Version 1.0 (Ingo Kofler)
Total: 94
ALU: 25
27 %
Jump: 8
9 %
Branch: 9
10 %
Memory: 19
21 %
Other: 33
35 %
Tool Control
Disconnect from MIPS
Reset
Close

1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo (Load-Use, etc.)	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	Load-Use y RAW	4 (2 de Load-Use y 2 del mul-addu)

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

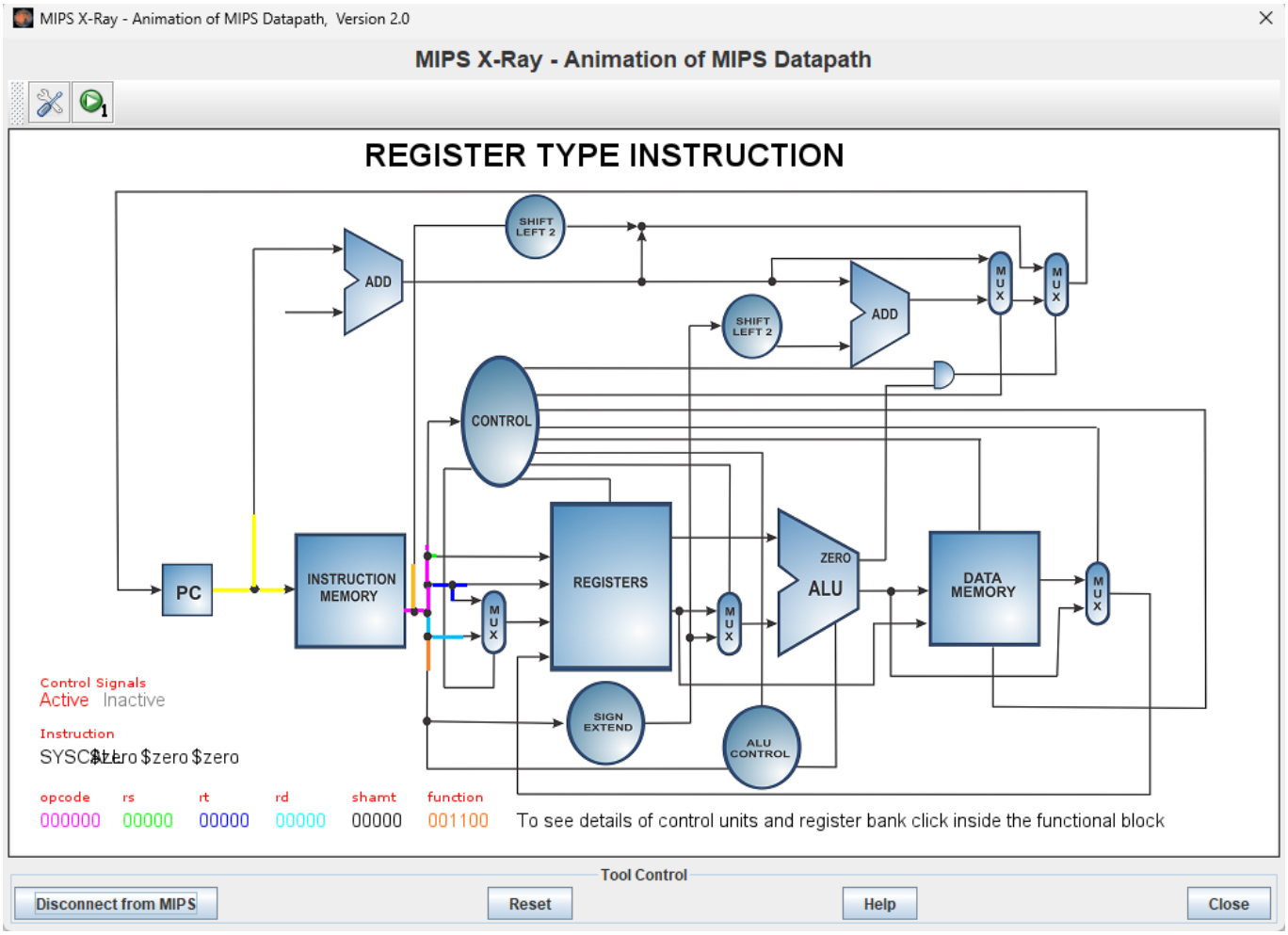
Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	95	94
Stalls (Paradas) por iteración	4 (2 de Load-Use y 2 del mul-addu)	0
Total de Stalls (8 iteraciones)	32 Stalls Totales	0
Ciclos Totales Estimados (Inst + Stalls)	95+32=127	94
CPI Estimado (Ciclos / Inst)	127/95≈1.34	1

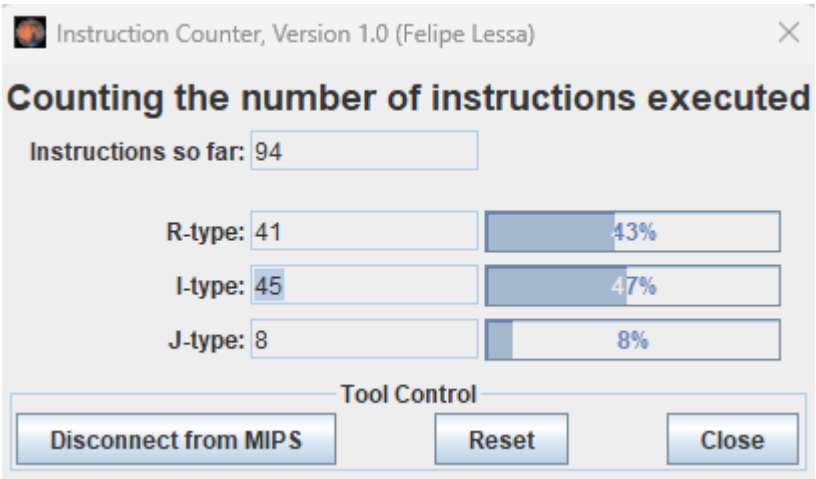
2. Optimización Propuesta

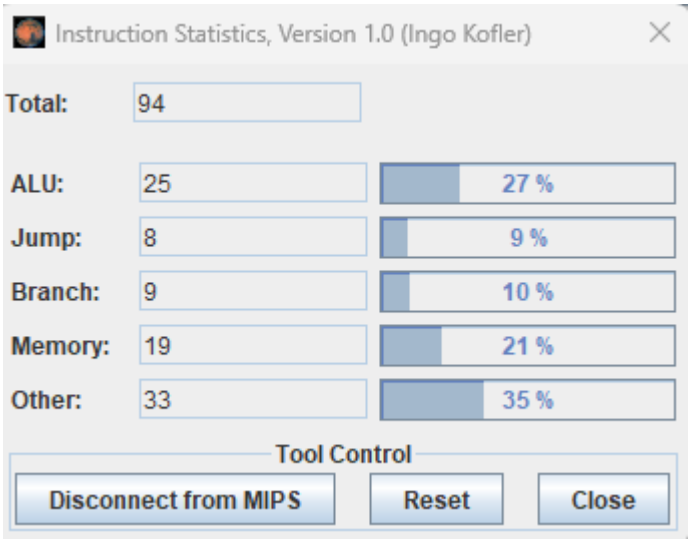
2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**.
- **Instruction Counter**.
- **Instruction Statistics**.


MIPS X-Ray - Animation of MIPS Datapath, Version 2.0
MIPS X-Ray - Animation of MIPS Datapath
REGISTER TYPE INSTRUCTION
SHIFT LEFT 2
ADD
ADD
SHIFT LEFT 2
MUX
MUX
CONTROL
PC
INSTRUCTION MEMORY
REGISTERS
ZERO
ALU
DATA MEMORY
MUX
MUX
MUX
SIGN EXTEND
ALU CONTROL
Control Signals
Active Inactive
Instruction
SYSCALL $zero $zero $zero
opcode rs rt rd shamt function
000000 00000 00000 00000 00000 001100
To see details of control units and register bank click inside the functional block
Tool Control
Disconnect from MIPS
Reset
Help
Close


Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 94
R-type: 41
43%
I-type: 45
47%
J-type: 8
8%
Tool Control
Disconnect from MIPS
Reset
Close


Instruction Statistics, Version 1.0 (Ingo Kofler)
Total: 94
ALU: 25
27 %
Jump: 8
9 %
Branch: 9
10 %
Memory: 19
21 %
Other: 33
35 %
Tool Control
Disconnect from MIPS
Reset
Close

2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin     # Si i == tamano, salir del bucle

    # --- Cálculo de dirección de memoria ---
    sll $t4, $t3, 2       # Desplazamiento: t4 = i * 4
    addu $t5, $s0, $t4    # t5 = dirección de X[i]

    # --- Carga de dato ---
    lw $t6, 0($t5)        # Leer X[i]

    # Optimizacion No. 1: se mueve addu $t9 aquí para eliminar el stall Load-Use
(lw - mul)
    addu $t9, $s1, $t4    # t9 = dirección de Y[i]

    # --- Operación aritmética ---
    mul $t7, $t6, $t0     # t7 = X[i] * A  (sin stall: $t6 queda listo de una vez)

    # Optimizacion 2: se mueve addi aquí para eliminar el stall mul-addu (mul -
addu)
    addi $t3, $t3, 1      # i = i + 1

    addu $t8, $t7, $t1    # t8 = t7 + B (sin stall: $t7 mismo caso al anterior
queda listo de una)

    # --- Almacenamiento de resultado ---
    sw $t8, 0($t9)        # Guardar resultado en Y[i]

    j loop

fin:
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

> [Tu explicación aquí]

---

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	127	94	26%
Stalls (Paradas)	32	0	100%

Métrica	Código Base	Código Optimizado	Mejora (%)
CPI	1.34	1.00	25.4%

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas?

> [Tus conclusiones aquí]